Objectives Assessment — PGSO System

Question answered: Does the project achieve the six stated objectives, and why?

Basis of assessment: Source-code review of the repository (`prisma/schema.prisma` , `src/app/**` , `src/lib/**` , `src/components/**`) as of **21 October 2026**. Runtime/database contents were not inspected; conclusions describe implemented capability, not the quality or completeness of live records.

---

1. Overall Verdict

#	Objective	Verdict	Short reason
1	Digitize receiving, inspection, inventory, repair monitoring, and document processing	**Fully met**	Complete Delivery → Inspection → IAR → Inventory → Issuance/Document chain exists, including automatic stock-in and generated/exportable forms.
2	Manage inventory and asset accountability (assignment, location, condition, maintenance history, responsible personnel)	**Fully met**	Asset records carry all required fields, transfers/assignments update custody, and a per-asset history combines requests, issuances, and repairs.
3	Track repair and maintenance activities (history, usability status, damaged assets)	**Fully met (minor caveat)**	Repair tickets, status workflow, technician/cost, receipts, and asset history exist. Usability is represented by `status` + free-text `condition` , not a dedicated controlled field.
4	Apply descriptive analytics through dashboards and reports (inventory, low stock, delayed transactions, workflow, repairs, asset conditions, quarterly issuance)	**Fully met**	Dashboards, stock health, workflow counts, repair/asset status, issuance trends, **aging/delayed-transaction analytics with a 7-day threshold**, a **calendar-quarter issuance report** (PAR/ICS, item, office, employee), and an **asset-condition breakdown** — on both portals, with official XLSX exports.
5	Implement prescriptive analytics (replenishment, replacement/disposal, reassignment of unused items, PPMP quantity suggestions)	**Partially met**	Rule-based replenishment and PPMP forecasting (low/base/high, budget-ceiling optimization) exist. **Replacement/disposal**, **idle-asset reassignment**, and **issuance-history-driven** quantity suggestions are missing.
6	Enhance asset tracking with QR-based identification and scanning	**Partially met**	Asset QR tags (URL payloads), QR display, and a request/issuance QR scanner exist. The in-app scanner cannot resolve **asset** QR tags; there is no scan-to-asset-action workflow.

Overall: The project **substantially achieves** the objectives. Three objectives are fully satisfied, and three are satisfied in part. Nothing in the six objectives is completely unaddressed. However, the system is best described as **descriptive + rule-based prescriptive decision support**, not a full predictive/prescriptive analytics platform. If the objectives are defended exactly as written, objectives 4, 5, and 6 need either additional features or narrower wording.

---

2. Detailed Assessment

Objective 1 — Digitize receiving, inspection, inventory recording, repair monitoring, and document processing

Verdict: Fully met.

What is implemented

Capability	Evidence in the codebase
Delivery/receiving log	`Delivery` and `DeliveryItem` models in `prisma/schema.prisma`; UI under `src/app/personnel/deliveries/`
Inspection and acceptance	`Inspection` and `IarRecord` models; `src/app/personnel/inspections/` including `[id]/inspection-form.tsx`; AIR generation
Automatic inventory recording	`stockInspectionItems()` in `src/lib/stock.ts` moves received quantities into `inventory` for `passed` and `partial` inspections, but only when an AIR/IAR backs the inspection. `stocked_qty` prevents double-counting.
Inventory management	`InventoryItem` model; `src/app/personnel/inventory/` with stock levels, thresholds, search, manual add/edit, and "Sync from inspections"
Repair monitoring	`Repair` model; `src/app/personnel/repairs/` with ticket creation, status flow, technician, cost, receipt, and reopen
Document processing	`Document` model plus dedicated views for delivery, inspection, stock, asset, repair, IAR, PAR, and ICS in `src/app/personnel/documents/page.tsx`
Printable/exportable forms	Receipt components; PAR/ICS Excel endpoints under `src/app/api/personnel/issuances/[id]/`; Excel helpers `src/lib/air-excel.ts`, `par-excel.ts`, `ics-excel.ts`, `asset-excel.ts`; "Download Excel" in `src/app/personnel/issuances/page.tsx`
Templates/imports	`src/lib/templates.ts` and asset import via `importAssetsFromExcel()` in `src/app/personnel/assets/actions.ts`

Explanation

The operational chain is genuinely digitized end to end. A delivery is logged, inspected, and accepted; accepted quantities automatically become stock; repairs are tracked as tickets; and the resulting records feed printable/exportable documents. This is not just data entry — the system enforces the receiving-to-stock and inspection-to-IAR relationships in `src/lib/stock.ts`.

Caveat

"Document processing" here means recording, generating, viewing, printing, and exporting documents. It does not appear to include full electronic signatures or a formal multi-level document approval workflow.

Objective 2 — Manage inventory and ensure asset accountability

Verdict: Fully met.

What is implemented

Required element	Implementation
Asset assignment	`Asset.assigned_to` and `Asset.end_user`; assignment executed on approved `new_assignment` requests in `src/app/personnel/requests/actions.ts`
Location	`Asset.location`; updated on approved transfers together with `end_user`
Condition	`Asset.condition` (create/edit forms and condition filters in `src/app/personnel/assets/`)
Maintenance history	`getAssetHistory()` in `src/app/personnel/assets/actions.ts` returns repairs, issuances, and requests for one asset; displayed by `[id]/AssetDetailEditor.tsx` and `asset-history.tsx`
Responsible personnel	Current holder (`assigned_to`) and end user, plus PAR/ICS issuance records in `issuance_records`
Inventory accountability	Quantities, units, unit cost, location, and reorder thresholds in `inventory`; stock is decremented on issuance/transfer

Explanation

Each asset is a first-class record, not a line in a spreadsheet. Custody is updated when a request is approved: transfers change `end_user` / `location`, new assignments set `assigned_to` / `end_user`, and stock picks decrement inventory. The asset detail page then reconstructs the asset's lifecycle from three independent sources — requests, issuances, and repairs — which is exactly what "maintenance history and responsible personnel" requires. PAR/ICS issuance documents provide the formal accountability trail.

Caveat

The system tracks the *current* holder and the *history of transactions*. It does not maintain a separate, explicit custodian-signature register beyond the generated PAR/ICS documents.

Objective 3 — Track repair and maintenance activities

Verdict: Fully met, with a minor caveat on "usability status."

What is implemented

- `Repair` model fields: `asset_id`, `reported_by`, `repair_date`, `description`, `status`, `cost`, `technician`, `created_at` (`prisma/schema.prisma`).
- Status workflow: `pending → in_progress → completed` (and reopen), enforced in `src/app/personnel/repairs/page.tsx` and its actions.
- Repair records are linked to the asset and appear in the asset history (`getAssetHistory`).
- Completed repairs produce a printable repair receipt (`repair-receipt.tsx`).
- Asset-level condition/status fields (`condition`, `status = available | in use | maintenance | retired`) capture damaged/out-of-service state.
- Repair requests from employees automatically create tickets when approved (`setRequestStatus` in `src/app/personnel/requests/actions.ts`).

Explanation

The system records the who/what/when/cost/technician of every repair and preserves the full ticket history rather than overwriting it. It can therefore answer "what was repaired, by whom, at what cost, and is it still open?" for any asset.

Caveat

"Usability status" is not a dedicated enumerated field such as `usable / repairable / unserviceable / for disposal`. It is inferred from `Asset.status` and the free-text `Asset.condition`. That is workable, but it is weaker for analytics and prescriptive rules (see Objective 5).

Objective 4 — Apply descriptive analytics through dashboards and reports

Verdict: Fully met.

What is implemented

Required analytic	Status	Evidence
Inventory status	Yes	Inventory page with SKUs, units on hand, low/critical/out counts; stock-level filters
Available and low-stock supplies	Yes	Low-stock watchlist and "Replenishment needed" alert in `src/app/personnel/dashboard/page.tsx` and `src/app/personnel/inventory/page.tsx`
Workflow status	Yes	Pending inspections, pending requests, active repairs, open documents on the personnel dashboard; request/repair/document status breakdowns in `src/app/super-admin/dashboard/analytics.ts`
Repair records	Yes	Repair counts by status and recent-repairs panels; repair list with filters
Asset conditions	Yes	Condition distribution from the shared registry (`getAssetConditionReport`), normalized with an explicit **Unspecified** bucket; serviceable/unserviceable headline; card on both dashboards and a full `/reports` tab (`src/lib/reporting-queries.ts` , `src/components/reporting/ReportsWorkspace.tsx`)
Delayed transactions	Yes	Aging report (`getAgingReport`) computes age for pending inspections (`deliveries.created_at`), pending requests (`requests.date_requested`), and open repairs (`repairs.created_at`), buckets them `0–3 / 4–7 / 8–14 / 15+` , flags records older than the uniform **7-day** threshold, and pages a filterable list. Counts appear on both dashboards
Quarterly supply issuance	Yes	`getQuarterlyIssuanceReport` groups `issuance_records` by calendar quarter for a selected year with PAR/ICS split, quantity, amount, 8-quarter trend, and top-items / by-office / by-employee breakdowns. Consolidated multi-line documents count once in the summary and per line in the item table
Dashboards and charts	Yes	Personnel dashboard stat cards, low-stock table, recent lists, monthly overview chart, plus aging and condition panels; Super Admin dashboard stat cards, deliveries-vs-inspections chart, and the analytics grid including delayed/aging, quarterly issuance, and condition cards

Required analytic	Status	Evidence
Reports and exports	Yes	`/personnel/reports` and `/super-admin/reports` provide filterable detail. Official ExcelJS workbooks are generated through `src/lib/report-excel.ts` and downloaded from `/api/{personnel,super-admin}/reports/{aging,quarterly-issuance,asset-conditions}` with period, filters, timestamp, and threshold metadata

Explanation

The descriptive layer now covers every item named in Objective 4. The personnel and Super Admin dashboards both surface inventory health, low-stock, workflow queues, repairs, asset status and condition, delayed/aging transactions, and quarterly issuance. The `/reports` workspace adds the filterable, exportable detail behind those summaries.

Scope is enforced consistently with the rest of the system: personnel see only their own deliveries, requests, repairs, and issuances; Super Admin sees system-wide records; the asset-condition pool is the shared registry for both. The safe-degradation path returns empty reports instead of failing when the scope or database is unavailable.

Conclusion: Objective 4 is fully met. Delayed transactions are measured in days with an auditable threshold, quarterly supply issuance is reported by quarter/item/office/employee, and asset conditions are summarized as a distribution with a documented limitation (no condition-history trend because the registry stores only the current condition).

Objective 5 — Implement prescriptive analytics

Verdict: Partially met.

What is implemented

Prescriptive capability	Status	Evidence
Recommend replenishment for low-stock items	Yes	`recommendationFor()` in `src/app/super-admin/inventory/page.tsx` returns “Replenish now / Restock soon / Plan reorder” with reasons; personnel inventory shows a replenishment alert and links to a `stock_replenishment` request (`src/app/personnel/requests/request-types.ts`)
Suggested quantities for PPMP preparation	Yes, but from uploaded PPMP history	`buildPlanningForecasts()` , `projectQtyHistory()` , `buildPpmpDraft()` in `src/lib/ppmp/analytics/planning.ts` produce low/base/high quantities using average, trend/linear regression, or growth; unit-cost escalation and multi-year horizon are applied
Budget-aware selection	Yes	`fitDraftToCeilingGreedy()` and `fitDraftToCeilingLP()` (0/1 knapsack via `javascript-lp-solver`) select/defer PPMP lines under a ceiling
Rule-based recommendations	Yes	`buildRecommendations()` in `src/lib/ppmp/analytics/rules.ts` : multi-year quantity change, above-average quantity, cost jumps, anomalies, low-confidence forecasts, and a `STOCK_COVER` rule using current inventory

Prescriptive capability	Status	Evidence
Inventory-aware PPMP review	Yes	`src/app/personnel/ppmp/analytics/actions.ts` builds an on-hand map and passes it into `runAnalysis()`
Replacement/disposal of damaged assets	**No**	No rule computes replacement or disposal recommendations from condition, status, repair frequency, or repair cost. `analytics.ts` only emits a generic text insight when `retired` is the top asset status
Re-assignment of items not in use	**No**	No idle/unused detection and no reassignment recommendation. Transfers are user-initiated requests, not system suggestions
Quantities based on inventory **and supply issuance history**	**Partial**	PPMP forecasting uses uploaded PPMP workbooks plus an inventory on-hand snapshot. `issuance_records` is not used to derive consumption or recommended quantities

Explanation

The strongest prescriptive component is the PPMP analytics/planning module. It is deterministic, auditable, and explains its calculations: every forecast row carries a method, assumptions, and evidence; recommendations carry rule IDs and calculation traces. The low-stock recommendations are also genuinely actionable because they link to the replenishment request flow.

The objective is still only partially met because two named prescriptions are absent:

- **Replacement or disposal of damaged assets.** The data needed exists (`condition` , `status` , repair count, repair cost), but no rule uses it to recommend "repair vs. replace vs. dispose."
- **Re-assignment of items not in use.** There is no query for assets marked `available` for a long period, assets assigned to inactive users, or assets with no issuance/repair activity, and no suggestion to transfer them.

In addition, the PPMP quantity suggestions are driven by **uploaded PPMP workbooks and on-hand inventory**, not by the system's own **supply issuance history**. If the objective specifically requires issuance-history-based quantities, that link is missing.

Conclusion: This is rule-based prescriptive analytics for replenishment and PPMP planning, not a complete prescriptive system covering replacement, disposal, and reassignment.

Objective 6 — Enhance asset tracking with QR-based identification and scanning

Verdict: Partially met.

What is implemented

Capability	Evidence
Unique asset identifier	`Asset.qr_code` is `@unique` ; `property_number` is also unique (`prisma/schema.prisma`)
QR generation	`src/lib/qrcode.ts` generates an SVG data URL. Asset tags encode `/personnel/assets/{id}` ; stock tags encode `/personnel/assets/stock/{id}`

Capability	Evidence
QR display/printing	QR card in `src/app/personnel/assets/[id]/AssetDetailEditor.tsx`; QR modal in `src/app/super-admin/assets/page.tsx`; QR data URL returned by `getAsset()` / `getUnifiedAssetsPage()`
QR scanner UI	`src/app/personnel/scan/page.tsx` and `src/components/QrScanner.tsx` support camera, image upload, and pasted payloads via `jsQR`
Scannable request/issuance records	`PGSO-REQUEST` and `PGSO-ISSUE` payloads are parsed and displayed with employee, item, quantity, dates, etc.
QR on issued documents	`RequestQrButton` and issue QR records in `src/app/personnel/issues/`

Explanation

Asset identification by QR is implemented: every asset has a unique QR/property number, the system generates a QR image that points to the asset's live detail page, and the QR is displayed for printing or attachment. Scanning that tag with a phone camera opens the asset page, which shows details and history. The in-app scanner also handles request and issuance QRs.

Gap

The in-app scanner's `parsePayload()` only recognizes `PGSO-REQUEST` and `PGSO-ISSUE`. It has **no branch for an asset URL or an asset payload**, so scanning a printed asset tag inside the scanner returns the raw URL text instead of resolving the asset. There is also no scan-driven action such as "scan asset → view / assign / log repair," and no batch or offline scanning. The roadmap in `src/lib/footer-content.ts` explicitly lists "offline-tolerant scanning" as future work.

Conclusion: QR-based asset **identification** is achieved; a fully integrated asset **scanning system** is partial.

---

3. Cross-Cutting Strengths

These do not map to a single objective but strengthen the claim that the project is operational:

- **Role separation:** employee, personnel, and super-admin areas with server-side guards (`src/lib/auth-guard.ts`, `src/lib/personnel-scope.ts`).
- **Audit trail:** `AuditLog` model and `writeAuditLog()` calls on key mutations.
- **Data integrity:** transactional stock-in, `stocked_qty` watermark, unique property/QR numbers, idempotency keys.
- **Usable at scale:** server-side pagination, debounced search, cached reads, optimistic updates.
- **Exports:** Excel workbooks for AIR, PAR, ICS, assets, and PPMP drafts.
- **Test coverage for analytics:** `src/__tests__/ppmp-analytics.test.ts`, `ppmp-planning.test.ts`, and related tests exercise the forecasting and recommendation logic.

---

4. Gaps That Prevent a "Fully Achieved" Rating

#	Gap	Affected objective	Why it matters
1	No replacement/disposal recommendation for damaged or repeatedly repaired assets	5	A named prescriptive action is missing
2	No idle/unused asset detection or reassignment recommendation	5	A named prescriptive action is missing
3	PPMP quantities are not derived from `issuance_records` (actual supply issuance history)	5	The objective explicitly says "based on inventory and supply issuance history"
4	In-app QR scanner does not resolve asset tags/URLs; no scan-to-action flow	6	Asset QR works with a phone camera, but the system's own scanning workflow is incomplete

> Objective 4's former gaps — days-pending/aging, the quarterly supply issuance report, and the asset-condition distribution — are now implemented (see Section 3) and are no longer counted as gaps.

---

5. Recommended Actions to Fully Satisfy the Objectives

Prioritized by effort-to-impact:

> **Objective 4 is now implemented.** The former actions — aging/delayed-transaction view, quarterly supply issuance report, and asset-condition analytics — shipped as `/personnel/reports`, `/super-admin/reports`, dashboard cards, and XLSX exports. The remaining actions below target Objectives 5 and 6.

1. **Add replacement/disposal and reassignment rules (Objective 5).** - Replacement/disposal: combine `condition` / `status`, repair count, cumulative repair cost, and asset age; emit "repair vs. replace vs. dispose" with a calculation trace. - Reassignment: flag assets `available` beyond N days, assigned to inactive users, or with no issuance/repair activity; suggest transfer targets. - Surface these in the existing recommendations panel so they share the advisory, evidence-based format.

2. **Feed issuance history into PPMP suggestions (Objective 5).** - Aggregate `issuance_records` by item/unit/quarter. - Use actual consumption as the baseline for low/base/high quantities, with inventory on-hand as a correction. - Keep uploaded PPMP workbooks as a comparison/validation source rather than the only history.

3. **Extend the QR scanner to asset tags (Objective 6).** - Recognize `/personnel/assets/{id}` URLs and an explicit `PGSO-ASSET` payload. - Resolve the asset and show details, condition, holder, and history. - Add actions: view asset, log repair, request transfer. - Optional: offline queue and batch scan.

4. **Introduce a controlled usability field (Objective 3, optional).** - Add `usability_status` (`serviceable`, `repairable`, `unserviceable`, `for_disposal`) to make reporting and prescriptive rules deterministic.

---

6. Suggested Objective Wording (If Features Are Not Added)

If the project will be defended without the gaps above, the objectives should be worded to match what is actually built:

- **Objective 4:** "...dashboards and reports that show inventory status, available and low-stock supplies, **pending workflow queues**, repair records, asset status, and **monthly issuance trends** for monitoring and evaluation."
- **Objective 5:** "Implement **rule-based prescriptive analytics** to recommend replenishment for low-stock items and to **forecast low/base/high quantities for annual PPMP preparation** using uploaded PPMP history and current inventory, with budget-ceiling optimization."
- **Objective 6:** "Enhance asset tracking by using **QR-coded asset tags that link to live asset pages**, together with a QR scanner for request and issuance records."

If the original wording must remain, implement the remaining gaps in Section 4 first.

---

7. Quick Verification Checklist (for Demo/Defense)

Use these paths to demonstrate each claim:

1. **Receiving → stock:** Personnel → Deliveries → log a delivery → Inspections → pass with AIR → Inventory shows the received quantity automatically.
2. **Accountability:** Employee → Requests → new assignment → Personnel approves/issuances → Asset detail shows `assigned_to`, `end_user`, location, and history.
3. **Repairs:** Personnel → Repairs → create ticket → start → complete with technician and cost → asset history and receipt reflect it.
4. **Descriptive analytics:** Personnel → Dashboard (queues, low stock, recent activity); Super Admin → Dashboard (status charts and insights).
5. **Prescriptive analytics:** Super Admin → Inventory (replenishment recommendation); Personnel → PPMP → Analytics (recommendations) → Planning (low/base/high, quarterly split, ceiling fit).
6. **QR:** Personnel → Assets → open an asset → QR card; scan with a phone camera to open the live page. Personnel → QR Scanner → scan a request/issuance QR.

---

8. Bottom Line

The project **achieves the core operational objectives** — digitized receiving, inspection, inventory, repairs, documents, asset accountability, repair history, and QR-based asset identification. It also has a credible descriptive dashboard layer and a rule-based PPMP prescriptive module.

It **fully achieves Objectives 1–4** and does **not yet fully achieve** Objectives 5–6 as literally written because:

- Replacement/disposal and idle-asset reassignment prescriptions are missing.
- PPMP quantities are not based on the system's own issuance history.
- The in-app QR scanner does not resolve asset tags.

> Objective 4 is now met: delayed transactions are reported with aging buckets and a 7-day threshold, quarterly supply issuance is reported by quarter/item/office/employee, and asset conditions are summarized as a distribution on both dashboards and in `/reports`.

Recommended position: Present the project as having **fully met Objectives 1–4** and **partially met Objectives 5–6**, with the remaining items documented as limitations/future work.